Programmation procédurale (UAA11)

Atelier : conditions et répétitives

Objectifs de l'atelier :

- ✔ Écrire une condition et éviter la confusion entre = et ==.
- ✔ Tester un intervalle et écrire le contraire d'une condition.
- ✔ Comprendre l'ordre d'une cascade de else if et le rôle du break dans un switch.
- ✔ Comparer while, for et do … while.
- ✔ Utiliser une boucle pour accumuler, contrôler une saisie et dessiner.

Avant de commencer

Cet atelier se fait seul, avant la séance de théorie. Tu n'as rien à apprendre par cœur : tu vas compiler de petits programmes, les exécuter, les modifier, et découvrir par toi-même comment le C se comporte. Les règles que tu formuleras serviront de point de départ à la séance.

Les programmes à télécharger :

https://lucasembrechts.github.io/UAA11/fichiers/atelier_conditions_repetitives.zip

Comment travailler :

- décompresse l'archive : tu obtiens un fichier `.c` par programme. Le nom du fichier est indiqué au-dessus de chaque programme ;
- ouvre chaque fichier dans ton éditeur habituel, compile-le et exécute-le ;
- **avant** chaque exécution, écris ta prédiction. Ne la corrige pas après coup : une prédiction fausse est justement ce qui va t'apprendre quelque chose ;
- certains programmes ne compilent pas : c'est voulu. Lis alors attentivement le message du compilateur, il fait partie de la découverte ;
- ne cherche pas la réponse dans le cours ou sur Internet : l'atelier est conçu pour que tu la trouves en observant ;
- à la fin de chaque section, résume ta découverte dans le cadre « Ma règle ». L'ensemble de tes règles formera ta fiche pour la séance théorique.

Section 0 : décider avec if

Fichier : section0.c

```
#include <stdio.h>

void main(void) {
    int note = 9;
    if (note >= 10) {
        printf("Reussi\n");
    } else {
        printf("Echec\n");
    }
    if (note = 10) {
        printf("Note parfaite ?\n");
    }
    printf("note vaut maintenant %d\n", note);
}
```

Prédis

La note vaut 9. Qu'affiche ce programme ?

Exécute, puis explique

Pourquoi « Note parfaite ? » est-il affiché ? Pourquoi `note` vaut-il maintenant 10 ?

Le compilateur t'a-t-il averti ?

Modifie

Corrige la deuxième condition. Qu'affiche maintenant le programme ?

✎ Ma règle : comment écrire une condition ?

Section 1 : tester un intervalle

Fichier : section1.c

```
#include <stdio.h>

void main(void) {
    int temperature = 35;
    if (18 <= temperature <= 25) {
        printf("Agreable\n");
    } else {
        printf("Pas agreable\n");
    }
}
```

Prédis

Il fait 35 degrés. Qu'affichera ce programme ?

Compile, exécute et observe

Que se passe-t-il chez toi ? Le compilateur dit-il quelque chose ?

Explique

Le C lit `18 <= temperature <= 25` de gauche à droite. Que vaut `18 <= temperature` ? Que compare-t-on ensuite à 25 ?

Modifie

Corrige la condition avec `&&`. Puis écris la condition contraire, « il ne fait PAS entre 18 et 25 degrés », sans utiliser `!`.

✎ Ma règle : comment tester qu'une valeur est comprise entre deux bornes ?

Section 2 : l'ordre des else if

Un jeu attribue un badge selon les points : bronze à partir de 50, argent à partir de 70, or à partir de 80.

Fichier : section2.c

```
#include <stdio.h>

void main(void) {
    int points = 85;
    if (points >= 50) {
        printf("Bronze\n");
    } else if (points >= 70) {
        printf("Argent\n");
    } else if (points >= 80) {
        printf("Or\n");
    }
}
```

Prédis, puis exécute

Le joueur a 85 points. Quel badge reçoit-il ?

Explique

85 est pourtant supérieur à 80. Pourquoi le programme n'affiche-t-il pas « Or » ?

Modifie

Remplace les deux `} else if` par `}` suivi d'un `if` indépendant. Qu'affiche le programme ?

Corrige le programme d'origine pour qu'il affiche le bon badge, en changeant seulement l'ordre des tests.

✎ Ma règle : comment fonctionne une cascade de else if ?

Section 3 : le switch et ses break

Fichier : section3.c

```
#include <stdio.h>

void main(void) {
    int choix = 2;
    switch (choix) {
        case 1:
            printf("Nouvelle partie\n");
        case 2:
            printf("Charger\n");
        case 3:
            printf("Options\n");
            break;
        default:
            printf("Choix inconnu\n");
    }
}
```

Prédis, puis exécute

Le joueur choisit 2. Qu'affiche ce programme ?

Explique

Pourquoi « Options » est-il affiché alors que le choix est 2 ? Que fait le `break` ?

Modifie

Ajoute les `break` manquants. Teste ensuite avec `choix = 7`. Qu'affiche le programme ?

Pourrait-on écrire `case choix >= 5:` dans un switch ? Teste.

✎ Ma règle : comment fonctionne un switch ?

Section 4 : trois façons de répéter

Fichier : section4.c

```c
#include <stdio.h>

void main(void) {
    int i = 0;
    while (i < 3) {
        printf("while %d\n", i);
        i++;
    }
    printf("apres le while, i = %d\n", i);
    for (int j = 1; j <= 3; j++) {
        printf("for %d\n", j);
    }
    int k = 10;
    do {
        printf("do %d\n", k);
        k++;
    } while (k < 3);
}
```

Prédis

Qu'affiche ce programme ? Attention au `do … while`.

Exécute, puis explique

Pourquoi `i` vaut-il 3 après le `while`, alors que la dernière valeur affichée est 2 ?

La condition `k < 3` est fausse dès le départ. Pourquoi le `do … while` s'exécute-t-il quand même une fois ?

Modifie

Fichier : section4_piege.c

```
#include <stdio.h>

void main(void) {
    for (int j = 1; j <= 3; j++);
    {
        printf("tour\n");
    }
}
```

Combien de fois « tour » sera-t-il affiché ? Exécute. Où est le piège ?

✎ Ma règle : quelles différences entre while, for et do … while ?

Section 5 : additionner jusqu'à un signal d'arrêt

Fichier : section5.c

```c
#include <stdio.h>

void main(void) {
    int note;
    int somme = 0;
    int nb = 0;
    printf("Note (-1 pour finir) : ");
    scanf("%d", &note);
    getchar();
    while (note != -1) {
        somme += note;
        nb++;
        printf("Note (-1 pour finir) : ");
        scanf("%d", &note);
        getchar();
    }
    printf("%d notes, moyenne %.2f\n", nb, (float) somme / nb);
}
```

Prédis, puis exécute

Tu vas taper 12, 15, 9, puis -1. Qu'affichera la dernière ligne ?

Explique

Pourquoi la note est-elle lue une première fois AVANT la boucle, puis à nouveau à la FIN de la boucle ?

Le -1 a-t-il été compté dans la somme ? Pourquoi ?

Pourquoi `somme` et `nb` sont-ils mis à 0 avant la boucle ?

Modifie

Exécute en tapant -1 dès le départ. Qu'affiche la dernière ligne ? Comment l'éviter ?

✎ Ma règle : comment répéter tant que l'utilisateur n'a pas donné le signal d'arrêt ?

Section 6 : redemander tant que c'est faux

Fichier : section6.c

```
#include <stdio.h>

void main(void) {
    int age;
    do {
        printf("Age (0 a 120) : ");
        scanf("%d", &age);
        getchar();
    } while (age < 0 || age > 120);
    printf("Age accepte : %d\n", age);
}
```

Prédis, puis exécute

Tape -5, puis 130, puis 17. Combien de fois la question est-elle posée ?

Explique

Pourquoi un `do … while` est-il bien adapté ici ?

Un âge valide est compris entre 0 et 120. Pourquoi la condition de la boucle utilise-t-elle `||` et pas `&&` ?

✎ Ma règle : comment obliger l'utilisateur à donner une valeur valide ?

Section 7 : une boucle dans une boucle

Fichier : section7.c

```
#include <stdio.h>

void main(void) {
    for (int ligne = 1; ligne <= 4; ligne++) {
        for (int col = 1; col <= ligne; col++) {
            printf("*");
        }
        printf("\n");
    }
}
```

Prédis, puis exécute

Dessine ce que le programme va afficher.

Explique

Combien de fois la boucle intérieure tourne-t-elle à la ligne 3 ? D'où vient ce nombre ?

Pourquoi le `printf("\n")` est-il placé après la boucle intérieure, mais dans la boucle extérieure ?

Modifie

Remplace `col <= ligne` par `col <= 5 - ligne`. Dessine le résultat avant d'exécuter.

✎ Ma règle : comment fonctionne une boucle imbriquée ?

Défi final

Crée un fichier `defi.c` : le jeu du nombre mystère.

- le nombre mystère est 37, défini avec `#define` ;
- le joueur a au plus 6 essais ; après chaque essai raté, le programme répond « Plus grand ! » ou « Plus petit ! » ;
- le jeu s'arrête dès que le nombre est trouvé ou que les 6 essais sont épuisés ;
- à la fin, le programme indique si le joueur a gagné (et en combien d'essais) ou perdu.

Deux exemples d'exécution :

```
Essai 1 : 50
Plus petit !
Essai 2 : 25
Plus grand !
Essai 3 : 40
Plus petit !
Essai 4 : 37
Bravo, trouve en 4 essais.
```

```
Essai 1 : 1
Plus grand !
...
Essai 6 : 6
Plus grand !
Perdu ! Le nombre etait 37.
```

Ta boucle a deux raisons de s'arrêter. Comment, après la boucle, sais-tu laquelle des deux s'est produite ?